UNIVERSIDAD NACIONAL DEL ALTIPLANO

Escuela Profesional de Ingeniería de Sistemas

GUÍA DE PRÁCTICA DE LABORATORIO — PRÁCTICA CALIFICADA 1

SEMANA 3

Herencia y Polimorfismo — Jerarquías, Sobrescritura y Principio de Sustitución

Semestre 2026-II · Práctica Calificada 3

Curso:	Programación Orientada a Objetos II — SIS206
Semana:	3 (Práctica Calificada 3)
Lenguaje:	Python 3.12+ / C++17
Duración:	2 horas de práctica (laboratorio)
Docente:	Dr. Aldo Hernán Zanabria Gálvez
ORCID / RENACYT:	0000-0003-3314-8768 | P0020608

Puno — Perú — 2026

I. OBJETIVO GENERAL

Diseñar e implementar jerarquías de herencia correctamente justificadas en Python y C++17, aplicando sobrescritura de métodos, polimorfismo dinámico y el principio de sustitución de Liskov (LSP), avanzando en el Proyecto Integrador del semestre.

1.1 Objetivos Específicos

- Identificar cuándo la herencia está justificada (relación IS-A) frente a cuándo debe preferirse la composición (HAS-A).
- Implementar jerarquías de herencia en Python usando super() y en C++ con herencia pública y virtual.
- Sobrescribir métodos con @override (Python 3.12+) y override (C++17), comprendiendo el mecanismo de vtable.
- Verificar el Principio de Sustitución de Liskov: todo objeto de una subclase debe poder sustituir a su clase base sin romper el comportamiento esperado.
- Extender el Proyecto Integrador con una jerarquía de herencia bien justificada.

II. MARCO TEÓRICO

2.1 Herencia — IS-A vs HAS-A

La herencia modela la relación "ES UN" (IS-A): un Docente ES UN PersonalUniversitario. La composición modela "TIENE UN" (HAS-A): un Docente TIENE UN Departamento. El error más frecuente es usar herencia cuando corresponde composición, creando jerarquías frágiles y difíciles de mantener (Larman, 2004):

```python
# ✅  Herencia justificada — IS-A
class PersonalUniversitario:
    def __init__(self, dni: str, nombres: str):
        self._dni    = dni
        self._nombres = nombres

    def identificarse(self) -> str:
        return f"{self._nombres} (DNI: {self._dni})"

class Docente(PersonalUniversitario):      # Docente ES PersonalUniversitario
    def __init__(self, dni, nombres, categoria: str):
        super().__init__(dni, nombres)
        self._categoria = categoria

    def calcular_bonificacion(self) -> float: ...


# ❌  Herencia NO justificada — misma interface, distintos dominios
class Automovil(Motor):    # Automovil NO es un Motor
    pass                    # debería ser composicion: self._motor = Motor()
```

2.2 Polimorfismo Dinámico y Sobrescritura

El polimorfismo dinámico permite que una referencia de tipo base invoque el método de la subclase real en tiempo de ejecución. En Python esto es automático; en C++ requiere declarar el método como virtual en la base y override en las subclases:

```python
# Python — polimorfismo dinámico automático
class PersonalUniversitario:
    def calcular_remuneracion(self) -> float:
        raise NotImplementedError("Subclase debe implementar este método")

class Docente(PersonalUniversitario):
    def calcular_remuneracion(self) -> float:
        return self._horas * self._tarifa * self._factor_categoria()

class Administrativo(PersonalUniversitario):
    def calcular remuneracion(self) -> float:
        return self._salario_base + self._bono_asistencia

# Lista heterogénea — polimorfismo en acción
personal: list[PersonalUniversitario] = [
    Docente("12345678", "Mamani", 16, "Asociado"),
    Administrativo("87654321", "Quispe", 2800.0),
]
for p in personal:
    print(f"{p.identificarse()}: S/ {p.calcular_remuneracion():.2f}")
```

```cpp
// C++17 — virtual + override
class PersonalUniversitario {
protected:
    std::string dni_;
    std::string nombres_;
public:
    PersonalUniversitario(std::string dni, std::string nombres)
```

```
        : dni_(std::move(dni)), nombres_(std::move(nombres)) {}

    virtual ~PersonalUniversitario() = default;  // OBLIGATORIO si hay virtual

    virtual float calcularRemuneracion() const = 0;  // método virtual puro

    virtual std::string identificarse() const {
        return nombres_ + " (DNI: " + dni_ + ")";
    }
};

class Docente : public PersonalUniversitario {
    int   horas_;
    float tarifa_;
public:
    Docente(std::string dni, std::string nombres, int horas, float tarifa)
        : PersonalUniversitario(std::move(dni), std::move(nombres)),
          horas_(horas), tarifa_(tarifa) {}

    float calcularRemuneracion() const override {
        return horas_ * tarifa_;
    }
};
```

2.3 Principio de Sustitución de Liskov (LSP)

El Principio de Sustitución de Liskov establece que si S es subtipo de T, entonces los objetos de tipo T en un programa pueden ser reemplazados por objetos de tipo S sin alterar ninguna propiedad deseable del programa (Liskov, 1987). En términos prácticos: una subclase no debe debilitar las precondiciones ni fortalecer las poscondiciones de la clase base:

```
# VIOLACIÓN del LSP — la subclase cambia el contrato
class Rectangulo:
    def set_ancho(self, v): self._ancho = v
    def set_alto(self,  v): self._alto  = v
    def area(self): return self._ancho * self._alto

class Cuadrado(Rectangulo):    # Cuadrado NO es Liskov-sustituto de Rectangulo
    def set_ancho(self, v):
        self._ancho = v
        self._alto  = v    # efecto secundario inesperado

# Código que funciona con Rectangulo pero FALLA con Cuadrado
def verificar_area(r: Rectangulo):
    r.set_ancho(4)
    r.set_alto(5)
    assert r.area() == 20   # falla si r es Cuadrado (area = 25)
```



Proyecto Integrador — Semana 3

Esta semana debe definir la jerarquía de herencia central del sistema. Identifique las clases base (abstractas o concretas) y al menos 2-3 subclases específicas del dominio. Verifique que cada subclase cumpla el LSP: ¿puede sustituir completamente a su clase base en cualquier contexto?

III. MATERIALES

Herramienta	Uso
Python 3.12+ / Colab	Actividades 1-3; verificación de LSP con assert
g++ -std=c++17 -Wall	Actividades 3-4; jerarquía con virtual y override
Draw.io / StarUML	Diagrama de herencia del Proyecto Integrador (Act. 5)
GitHub	Commit: "feat(semana3): jerarquía de herencia"

IV. INSTRUCCIONES — PC1

⚠ **Esta es la Práctica Calificada 1 (PC1)**
La nota de esta sesión corresponde a la Práctica Calificada 1 del semestre (peso: 30% dentro de la Unidad I). Trabaje de forma individual. Al finalizar, suba su notebook a Google Classroom y haga commit al repositorio GitHub. Tiempo: 2 horas.

1. Abrir el notebook de Colab de las semanas anteriores y agregar una sección "Semana 3 ".
2. Completar las actividades en orden; cada actividad tiene criterios de calificación explícitos.
3. El código debe ejecutarse sin errores. Los bloques TODO deben estar completamente implementados.
4. La Actividad 5 se continúa fuera de clase: commit al repositorio antes del siguiente lunes.

V. DESARROLLO DE ACTIVIDADES

ACTIVIDAD 1 — Análisis — Identificar IS-A vs HAS-A

Python + C++

Para cada relación propuesta, indique si debe implementarse con HERENCIA (IS-A) o con COMPOSICIÓN (HAS-A), justificando brevemente. Si sugiere herencia, especifique la clase base y la subclase.

Relación propuesta	IS-A / HAS-A	Justificación
Un Estudiante de Posgrado en relación a un Estudiante		
Un Sensor IoT en relación a un Dispositivo		
Un Informe en relación a una Impresora		
Un Libro Digital en relación a un Libro		
Una Motocicleta en relación a un Vehículo		
Un Departamento en relación a un Empleado		

ACTIVIDAD 2 — Jerarquía de Personal — Python con polimorfismo

Python + C++

Complete la implementación de la jerarquía de personal universitario. La clase base PersonalUniversitario ya está implementada; usted debe completar las subclases Docente y Administrativo. Luego implemente la función planilla() que demuestra el polimorfismo.

```python
# ── Práctica S3 — Actividad 2: Jerarquía PersonalUniversitario ──────────
from abc import ABC, abstractmethod
from dataclasses import dataclass


class PersonalUniversitario(ABC):
    """Clase base abstracta para todo el personal de la UNAP."""
    TARIFA_HORA_BASE = 75.0   # S/ por hora

    def __init__(self, dni: str, nombres: str, apellidos: str):
        self.__dni      = self._validar_dni(dni)
        self._nombres   = nombres
        self._apellidos = apellidos

    @staticmethod
    def _validar_dni(dni: str) -> str:
        if not (dni.isdigit() and len(dni) == 8):
            raise ValueError(f"DNI inválido: {dni!r} (debe tener 8 dígitos)")
        return dni

    @property
    def dni(self) -> str: return self.__dni

    @property
    def nombre_completo(self) -> str:
        return f"{self._apellidos}, {self._nombres}"

    @abstractmethod
    def calcular_remuneracion(self) -> float:
        """Cada subclase define su propio cálculo."""
        ...

    def __str__(self) -> str:
        return f"{self.nombre_completo}  →  S/ {self.calcular_remuneracion():.2f}"


class Docente(PersonalUniversitario):
    FACTORES = {"Auxiliar": 1.0, "Asociado": 1.4, "Principal": 1.8}

    def __init__(self, dni: str, nombres: str, apellidos: str,
                 horas: int, categoria: str):
        super().__init__(dni, nombres, apellidos)
        # TODO: validar horas > 0 y <= 40
        # TODO: validar categoria en FACTORES
        # TODO: asignar self._horas y self._categoria
        ...

    def calcular_remuneracion(self) -> float:
        # TODO: horas * TARIFA_HORA_BASE * factor de categoria
        ...


class Administrativo(PersonalUniversitario):
    def __init__(self, dni: str, nombres: str, apellidos: str,
                 salario_base: float, bono: float = 0.0):
        super().__init__(dni, nombres, apellidos)
        # TODO: validar salario_base > 0
```

```
        # TODO: asignar self._salario_base y self._bono
        ...

    def calcular_remuneracion(self) -> float:
        # TODO: salario_base + bono
        ...


class Investigador(PersonalUniversitario):
    """Investigador a tiempo completo — remunerado por proyectos activos."""
    REMUNERACION_POR_PROYECTO = 1500.0

    def __init__(self, dni: str, nombres: str, apellidos: str,
                 proyectos: int):
        super().__init__(dni, nombres, apellidos)
        if proyectos < 1:
            raise ValueError("Debe tener al menos 1 proyecto activo")
        self._proyectos = proyectos

    def calcular_remuneracion(self) -> float:
        return self._proyectos * self.REMUNERACION_POR_PROYECTO


def planilla(personal: list[PersonalUniversitario]) -> None:
    """Imprime la planilla usando polimorfismo — sin isinstance()."""
    total = 0.0
    print("=" * 60)
    print(f"PLANILLA PERSONAL UNIVERSITARIO — UNAP PUNO")
    print("=" * 60)
    for p in personal:
        print(p)       # llama __str__ → calcular_remuneracion polimórfico
        total += p.calcular_remuneracion()
    print("-" * 60)
    print(f"TOTAL PLANILLA:  S/ {total:.2f}")


# ── Pruebas ──────────────────────────────────────────────────────
personal = [
    Docente("12345678", "Juan Carlos", "Mamani Quispe",   16, "Asociado"),
    Docente("23456789", "María Elena", "Quispe Torres",   12, "Principal"),
    Administrativo("34567890", "Pedro", "Huanca Lazo",    2800.0, 350.0),
    Investigador("45678901", "Ana Lucía", "Flores Ramos", 2),
]
planilla(personal)

# Verificar LSP — todos son PersonalUniversitario
for p in personal:
    assert isinstance(p, PersonalUniversitario)
    assert p.calcular_remuneracion() > 0
print("\nVerificación LSP: OK — todos los objetos respetan el contrato base")
```

ACTIVIDAD 3 — Verificación del LSP — Test de sustitución

Python + C++

Implemente la función verificar_lsp() que, dado cualquier objeto PersonalUniversitario, ejecuta un contrato invariante sobre él. Luego demuestre que TODAS las subclases lo pasan. Finalmente, implemente la jerarquía Figura → Rectangulo → Cuadrado y explique por qué Cuadrado VIOLA el LSP.

```
# ── Verificación LSP para la jerarquía PersonalUniversitario ──────────
def verificar_contrato_personal(p: PersonalUniversitario) -> bool:
    """
    Contrato que TODA subclase debe satisfacer:
    1. nombre_completo retorna string no vacío
    2. calcular_remuneracion() retorna valor > 0
    3. str(p) contiene el nombre completo
    """
    ok = True
    ok = ok and isinstance(p.nombre_completo, str) and len(p.nombre_completo) > 0
    ok = ok and p.calcular_remuneracion() > 0
    ok = ok and p.nombre_completo in str(p)
    return ok


# TODO: ejecutar verificar_contrato_personal sobre cada objeto de personal[]
# Debe imprimir OK para todos


# ── Violación clásica de LSP: Cuadrado hereda de Rectangulo ────────────
class Figura:
    def area(self) -> float: ...

class Rectangulo(Figura):
    def __init__(self, ancho: float, alto: float):
        self._ancho = ancho
        self._alto  = alto

    def set_ancho(self, v: float) -> None: self._ancho = v
    def set_alto(self,  v: float) -> None: self._alto  = v
    def area(self) -> float: return self._ancho * self._alto

class Cuadrado(Rectangulo):
    def __init__(self, lado: float):
        super().__init__(lado, lado)

    def set_ancho(self, v: float) -> None:
        self._ancho = v
        self._alto  = v    # fuerza ambos lados iguales

    def set_alto(self, v: float) -> None:
        self._ancho = v
        self._alto  = v


def test_rectangulo(r: Rectangulo) -> None:
    """Test que DEBERÍA pasar para cualquier Rectangulo según LSP."""
    r.set_ancho(4)
    r.set_alto(5)
    area_esperada = 20.0
    assert r.area() == area_esperada, f"LSP violado: area={r.area()},
esperado={area_esperada}"
```

```
rect = Rectangulo(2, 2)
test_rectangulo(rect)   # OK

cuad = Cuadrado(2)
try:
    test_rectangulo(cuad)   # FALLA — viola LSP
except AssertionError as e:
    print(f"LSP VIOLADO en Cuadrado: {e}")
    print("Solución: no heredar Cuadrado de Rectangulo; usar composición o interfaz independiente.")
```

Pregunta de análisis (escriba su respuesta abajo):

¿Por qué Cuadrado viola el LSP aunque matemáticamente un cuadrado ES un rectángulo? ¿Qué alternativa de diseño propone para modelar ambas figuras sin violar el principio?

ACTIVIDAD 4 — Jerarquía en C++17 — virtual, override y destructor

Python + C++

Implemente en C++17 la jerarquía de personal universitario equivalente a la Actividad 2. Debe incluir: destructor virtual en la base, método virtual puro calcularRemuneracion(), polimorfismo con vector de unique_ptr, y la función planilla().

```
// ── personal_unap.hpp — Práctica S3, Actividad 4 ──────────────────────
#pragma once
#include <string>
#include <vector>
#include <memory>
#include <stdexcept>
#include <iostream>
#include <iomanip>

class PersonalUniversitario {
protected:
    std::string dni_;
    std::string nombres_;
    std::string apellidos_;

public:
    PersonalUniversitario(std::string dni,
                          std::string nombres,
                          std::string apellidos)
        : dni_(std::move(dni)),
          nombres_(std::move(nombres)),
          apellidos_(std::move(apellidos)) {}

    virtual ~PersonalUniversitario() = default;  // destructor virtual OBLIGATORIO
```

```
    virtual float calcularRemuneracion() const = 0;  // método virtual puro

    virtual std::string identificarse() const {
        return apellidos_ + ", " + nombres_ + " (DNI: " + dni_ + ")";
    }
};


class Docente : public PersonalUniversitario {
    int   horas_;
    float tarifa_ = 75.0f;
    float factor_;

    static float resolverFactor(const std::string& cat) {
        if (cat == "Auxiliar")  return 1.0f;
        if (cat == "Asociado")  return 1.4f;
        if (cat == "Principal") return 1.8f;
        throw std::invalid_argument("Categoria invalida: " + cat);
    }

public:
    Docente(std::string dni, std::string nombres, std::string apellidos,
            int horas, const std::string& categoria)
        : PersonalUniversitario(std::move(dni), std::move(nombres),
                                std::move(apellidos)),
          horas_(horas), factor_(resolverFactor(categoria)) {
        if (horas <= 0 || horas > 40)
            throw std::invalid_argument("Horas fuera de rango [1,40]");
    }

    float calcularRemuneracion() const override {
        return horas_ * tarifa_ * factor_;
    }
};


class Administrativo : public PersonalUniversitario {
    float salarioBase_;
    float bono_;
public:
    Administrativo(std::string dni, std::string nombres, std::string apellidos,
                   float salarioBase, float bono = 0.0f)
        : PersonalUniversitario(std::move(dni), std::move(nombres),
                                std::move(apellidos)),
          salarioBase_(salarioBase), bono_(bono) {
        if (salarioBase_ <= 0)
            throw std::invalid_argument("Salario base debe ser positivo");
    }

    float calcularRemuneracion() const override {
        return salarioBase_ + bono_;
    }
};


void planilla(const std::vector<std::unique_ptr<PersonalUniversitario>>& p) {
```

```
    float total = 0.0f;
    std::cout << std::string(60, '=') << "\n";
    std::cout << "PLANILLA PERSONAL UNIVERSITARIO — UNAP PUNO\n";
    std::cout << std::string(60, '=') << "\n";
    for (const auto& persona : p) {
        std::cout << std::fixed << std::setprecision(2)
                  << persona->identificarse()
                  << "  S/ " << persona->calcularRemuneracion() << "\n";
        total += persona->calcularRemuneracion();
    }
    std::cout << std::string(60, '-') << "\n";
    std::cout << "TOTAL:  S/ " << total << "\n";
}


// main.cpp
int main() {
    std::vector<std::unique_ptr<PersonalUniversitario>> personal;
    personal.push_back(std::make_unique<Docente>(
        "12345678","Juan Carlos","Mamani Quispe", 16,"Asociado"));
    personal.push_back(std::make_unique<Administrativo>(
        "34567890","Pedro","Huanca Lazo", 2800.0f, 350.0f));
    planilla(personal);
    return 0;
}
```

ACTIVIDAD 5 — Proyecto Integrador — Jerarquía de herencia del sistema

Python + C++

Aplique herencia al Proyecto Integrador definido en las Semanas 1 y 2. Esta actividad extiende el código ya entregado.

- Identifique y justifique al menos 1 clase base y 2 subclases IS-A en su dominio. Complete la tabla:

Clase base	Subclase 1	Subclase 2
Relación IS-A verificada:		

- Implemente la jerarquía en Python y C++17 (con virtual/override en C++). Cada subclase debe sobrescribir al menos 1 método relevante del dominio.
- Escriba una función que itere sobre una lista heterogénea de objetos de la clase base y demuestre el polimorfismo dinámico.
- Actualice el diagrama de clases UML en Draw.io mostrando la jerarquía con la flecha de herencia (▷ hueca) y las relaciones de composición de las semanas previas.
- Commit: "feat(semana3): jerarquía de herencia y polimorfismo" al repositorio GitHub.

VI. PREGUNTAS DE REFLEXIÓN

1. En su Proyecto Integrador, ¿encontró alguna situación donde pensó en usar herencia pero al final eligió composición (o viceversa)? Describa el caso y justifique la decisión.

2. ¿Por qué en C++ es obligatorio que el destructor de la clase base sea virtual si se usa polimorfismo dinámico? ¿Qué ocurre exactamente si no se declara virtual y se destruye un objeto a través de un puntero a la base?
3. El polimorfismo en Python es "duck typing": cualquier objeto que tenga el método correcto puede usarse, sin necesidad de herencia formal. ¿Cuáles son las ventajas y desventajas de este enfoque frente al polimorfismo de C++ basado en tipos estrictos?

VII. RÚBRICA — PC1

Criterio	Excelente (4)	Bueno (3)	Regular (2)	Insuf. (1)	Peso
Análisis IS-A vs HAS-A (Act. 1)	6 de 6 correctas con justificación técnica.	4-5 correctas.	2-3 correctas.	<2 o no entrega.	**15%**
Jerarquía Python (Act. 2)	Ejecutable; planilla() polimórfica; validaciones en subclases.	Ejecutable con errores menores.	Parcialmente implementado.	No ejecuta.	**25%**
Verificación LSP (Act. 3)	Contrato completo; análisis correcto de violación Cuadrado; alternativa propuesta.	Contrato parcial; análisis superficial.	Solo reproduce el código.	No responde.	**20%**
Jerarquía C++17 (Act. 4)	Compila -Wall; destructor virtual; override; unique_ptr; planilla().	Compila con warnings menores.	Compila sin override o sin virtual.	No compila.	**20%**
Proyecto integrador (Act. 5)	Jerarquía Python+C++; polimorfismo demostrado; UML actualizado; commit.	Solo Python; sin UML.	Jerarquía sin polimorfismo.	No entrega.	**20%**

VIII. REFERENCIAS

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns. Addison-Wesley.

Larman, C. (2004). Applying UML and Patterns (3rd ed.). Prentice Hall.

Liskov, B. (1987). Data abstraction and hierarchy. ACM SIGPLAN Notices, 23(5). https://doi.org/10.1145/62138.62141

Martin, R. C. (2008). Clean Code. Prentice Hall.

cppreference.com. (2026). virtual function specifier. https://en.cppreference.com/w/cpp/language/virtual